CAJEROS AUTOMÁTICOS

LUCAS RUIZ,RAFAEL BRITO

Contenido

Introducción 2

Conclusión 3

Bibliografía 3

Análisis

El escenario escogido es una aplicación que estaría simulando transacciones bancarias en diferentes cajeros automáticos de la misma institución financiera. En este sistema, el recurso principal compartido es la base de datos centralizada del banco, específicamente el registro del saldo de los clientes. Esto requiere una cooperación y sincronización importante los diversos cajeros podrían intentar acceder y modificar el saldo de una misma cuenta entonces si no se coordina de forma adecuada, se podría generar condiciones de carrera, dando en consecuencia inconsistencias graves como retirar más fondos de los disponibles o perder el registro de un depósito.

Tabla comparativa: implementación de modelos

Característica	**Modelo de Memoria Compartida**	**Modelo de Paso de Mensajes**
Arquitectura en este escenario	Todos los cajeros accederán directamente a un espacio de memoria común donde se almacenan los saldos de las cuentas.	Los cajeros funcionan como clientes independientes que envían solicitudes (mensajes) a un servidor central del banco.
Sincronización	Requiere el uso explícito de mecanismos de control de concurrencia en cada cajero para bloquear la cuenta mientras se realiza la transacción.	La sincronización está implícita en los mensajes. El servidor central recibe las peticiones en una cola y las procesa de forma estructurada.
Ventajas	Alta velocidad de acceso a los datos, ya que no hay sobrecarga por el envío de mensajes a través de una red.	Escalable y natural para sistemas distribuidos. Aísla la lógica de negocio; si un cajero falla, no corrompe la memoria del servidor.
Desventajas	Complejidad alta para evitar bloqueos mutuos. Es muy difícil y riesgoso de implementar en sistemas físicamente distribuidos como una red real de cajeros.	Existe una ligera latencia debido al tiempo de comunicación por red entre el cajero y el servidor central.

Planificación

Dentro de la arquitectura de los cajeros automáticos el recurso principal sería el registro del saldo de la cuenta bancaria. Representando el valor monetario que tiene disponible para poder realizar transacciones financieras en distintos puntos de atención. Teniendo en cuenta que múltiples unidades de ejecución podrían intentar consultar o alterar el saldo de una cuenta de forma paralela, es importante coordinar el acceso a dicho valor. Al existir la ausencia de mecanismos de control y sincronización, derivaría en la sobreescritura de datos, pérdida de registros de depósitos o la autorización de retiros por encima del saldo real.

Formas de representación del recurso compartido en python

Característica	**Representación de tipo básico**	**Representación de tipo complejo**
Estructura en python	Número entero (int) o de punto flotante (float).	Diccionario (dict) u objeto estructurado que contiene múltiples campos.
Descripción del recurso	Almacena únicamente el valor numérico puro que representa la cantidad del saldo actual (por ejemplo: saldo = 1500.00)	Almacena el saldo acompañado de metadatos esenciales como el identificador de la cuenta, estado y un historial de movimientos (por ejemplo: {"id": "ACC-01", "saldo": 1500.00, "historial": []}).

Diseño

Se diseñaron cuatro clases principales que gestionan la interacción, las unidades de ejecución compartido. La siguiente tabla detalla la responsabilidad, atributos y métodos de cada clase:

Clase	Responsabilidad	Atributos	Métodos
Cuenta	Representar y encapsular el recurso compartido (saldo), asegurando que cualquier cambio mantenga la integridad de los fondos.	id_cuenta saldo estado	consultar_saldo() depositar(monto) retirar(monto)
Cajero	Simular el punto físico de atención donde el usuario interactúa, generando los hilos de ejecución independientes.	id_cajero ubicacion	procesar_operacion() enviar_mensje_servidor()
Usuario	Representar al cliente titular que interactúa con el cajero para acceder a sus cuentas bancarias.	id_usuario nombre cuentas_asociadas	iniciar_sesion() seleccionar_cuenta()
Transacciones	Modelar la operación financiera solicitada (retiro, depósito o consulta) que viaja como mensaje.	id_transaccion tipo monto id_cuenta	validar_transaccion()

Las unidades de ejecución (hilos mediante threading.Thread) están asociadas a la clase Cajero, permitiendo que múltiples puntos de atención funcionen de manera concurrente. Cada hilo de cajero genera instancias de la clase Transacciones en nombre del Usuario, las cuales son enviadas de forma estructurada para modificar o consultar la clase Cuenta a través del servidor.

Sección crítica

Esta sección comprende el bloque de operaciones donde las unidades de ejecución consultan y modifican los datos financieros del recurso compartido. Se estaría analizando 3 escenarios, sus problemas asociados a su ejecución simultánea y el mecanismo de sincronización propuesto.

- **Transacciones al mismo tiempo**

  Los principales problemas con este escenario serían las condiciones de carrera, lectura de saldos desactualizados y sobreescritura de datos que pueden llevar a entregar fondos inexistentes.

- **Acceder a cuentas diferentes del mismo usuario a la vez**

  Al intentar acceder a cuentas diferentes del mismo usuario a la vez ocurrirán conflictos de concurrencia a nivel de sesión o bloqueos cruzados si el sistema no aísla las operaciones por identificador de cuenta de manera independiente.

- **Acceder al banco al mismo tiempo**

  Si todos los cajeros envían solicitudes masivas al servidor provocan una saturación de este mismo, causando pérdidas de peticiones y corrupción en la base de datos por falta de ordenamiento de peticiones.

Teniendo en cuenta que el modelo seleccionado es el paso de mensajes, no se requieren bloqueos directos en la memoria compartida. En su lugar, el servidor central implementa una cola de mensajes FIFO. Cuando se presentan transacciones simultáneas de una en una y las encola de forma secuencial y atómica sobre la cuenta, garantizando que el recurso se mantenga totalmente consistente y evitando cualquier condición de carrera sin necesidad de sincronización compleja a nivel de hardware.

Tabla comparativa de la representación del recurso

Característica	**Tipo Básico**	**Tipo Complejo**
Descripción	Un número entero (o flotante) que representa una cantidad o un contador del saldo.	Una lista o un diccionario que permite registrar información adicional de la cuenta.

Ventajas	Máxima simplicidad en la codificación y rapidez en las operaciones aritméticas directas.	Aporta trazabilidad, contexto y permite modelar un entorno bancario realista con múltiples cuentas.
Desventajas	Imposible de utilizar en un sistema real con múltiples cuentas de distintos usuarios por falta de identificación.	Requiere una estructura y manipulación de datos más compleja dentro del código.

Para la implementación del proyecto se utilizará la representación de tipo complejo. En la arquitectura propuesta de cajeros automáticos con paso de mensajes, las solicitudes enviadas por los cajeros al servidor central no pueden operar sobre un número aislado. Es un requisito indispensable que cada transacción viaje acompañada de metadatos (como el identificador de la cuenta, el tipo de operación y el monto) para que el servidor central pueda identificar con precisión absoluta cuál cuenta debe consultar o modificar. Utilizar un tipo complejo garantiza la consistencia de los datos, evita conflictos entre cuentas y da soporte al modelo distribuido de la institución financiera.

Conclusiones

Como conclusión se tiene que el modelo de pasos de mensajes tiende a ser más efectivo en situaciones realistas a la hora de gestionar la coordinación de los cajeros automáticos. Debido a la ubicación de estos cajeros, usar un esquema de memoria compartida tiende a ser ineficiente a nivel de hardware y presenta un riesgo de seguridad grave en la integridad de datos. El paso de mensajes permite que cada cajero actúe como un nodo independiente que envía mensajes a un servidor centralizado. Este servidor estaría actuando como un portero que procesa transacciones de forma segura, garantizando las sincronizaciones con una buena consistencia.

Bibliografía

***Inter Process Communication tutorial*. (s. f.).**

https://www.tutorialspoint.com/inter_process_communication/index.htm

GeeksforGeeks. (2023). ***Difference between shared memory and message passing model in operating system*****. GeeksforGeeks.** https://www.geeksforgeeks.org/difference-between-shared-memory-and-message-passing-model-in-operating-system/

Inter Process Communication tutorial**. (s. f.).** https://www.tutorialspoint.com/inter_process_communication/index.htm

Methods in Interprocess Communication**. (s. f.).** https://www.tutorialspoint.com/article/methods-in-interprocess-communication